Sere D., Alina J., Gabby P.
CSCI 2113 - Dr. Kinga
Final Project Requirements

Tic Tac Toe online multiplayer game

#	Description	Functional /non functional	Testing input	Expected output	Pass
1	JButton connect	Functional	User enters correct ip address and port number then clicks connect	We can see that both players have successfully joined the server once the game says who is player X and player O after assigning it and displaying it in the JTextArea made for indicating which symbol the user is assigned. Additionally we can see that GameClient.java has correctly inherited an instance of gameGUI.java which interacts with tictactoeboard.java	
2	updateboard()	Functional	A user makes a move by clicking on of the spots on the 3x3 board	Both GUIs for the clients are updated appropriately after move is made - the spot chosen by user is filled with X or O and GUI displays who's turn it is next in JTextArea dedicated to that	
3	restboard()	Functional	User clicks the reset game JButton OR the new game JButton	Clear all the input that was loaded into the 2D array that stores info about tictactoeboard based on moves made by users during gameplay. Reset all elements in the 2d	

				array to be 0 (null/ clickable)instead of 1 or 2 (for filled with either X or O)	
4	Boolean makeMove()	Functional	User makes a move by clicking an open space on the 3x3 board	if you make a move update this boolean to true (and then also update the 2D array in tictactoeboard.java to store this move in game)	
5	checkWin()	Functional	User plays the game and gets 3 X or O symbols in a row. Then this method should evaluate the win and say which player it is that won	The JTextArea on GUI is updated to either “player X wins” or “player O wins”	
6	checkDraw()	Functional	Users play the game in such a way that all spaces are filled but no 3-in-a-row combinations for X or O are found in the board	The JTextArea on GUI is updated to “No Winners, Game is a Draw”	
7	displayEndGame	Functional	Users play game until completion (and a win or draw is declared after)	Make sure at the end it display certain person has lost/won and prompts user to start a new game through use of “new game” JButton	
8	lastMove()	Functional	Users make a move on the game board	The JTextArea that displays the last move played is updated to show which cell in the 3x3 board was selected and by which player	
9	Reading thread keeps taking	Non functional	Keep a count to see how long it takes for each player to receive	Keep track of play move, insure that in 10 sec that the player are	

	incoming messages/moves from other players		the other player's move after the other player has completed their turn	being communicated each move the opposite player is making via server updates to their local GUIs	
10	Public void server()	Non functional	Users are able to connect and disconnect from the server in 10 sec.	The player is continuously connected to the server as long as the server is running. If you disconnect from the server you disconnect from the game and have no updates to your local GUI on your JFrame anymore.	

HERE'S A HIGH-LEVEL OVERVIEW OF HOW THE UML CLASSES INTERACT:

Game Initialization:

- An instance of GameServer is created to manage the game.
- Multiple instances of GameClient connect to the server.
- Each GameClient has its own instance of GameGUI for the local interface.

Game Loop:

- When a player makes a move on a local GameClient, the move is sent to the GameServer.
- The GameServer updates its instance of TicTacToeBoard based on the move.
- The GameServer then sends updates to all connected GameClient.
- Each GameClient updates its local GameGUI based on the received updates.

End of Game:

- If the game reaches a win, loss, or draw condition, the GameServer sends this information to all connected GameClient.
- Each GameClient updates its local GameGUI to reflect the end of the game.

EXPLANATION OF UML DIAGRAM:

- GameServer depends on info from GameClients
- GameClients depend on info from TicTacToeBoard so they can update their local GUIs
- TicTacToeBoard depends on GameServer because it'll update game state based on info from moves made by either client connected to the server
- GameGUI is inherited by GameClient because each player will have a local GUI to interact with. This local GUI will also be updated by the server (which, as mentioned above, gets info from TicTacToeBoard - TicTacToeBoard is updated with moves made by either player connected to the server)

PROJECT CONCEPTUALIZATION AND CLASS RESPONSIBILITIES:

GameServer.java (Server Class):

- Responsibilities:
    - Manages the overall game state and logic.
    - Listens for incoming connections from clients.
    - Handles the initiation of a new game, including assigning symbols to players. (first one to get on the server is x and last person will be o)
    - Coordinates and updates clients about the game state.
    - Checks for win/draw conditions and updates the clients accordingly.
- Why?
    - Encapsulation: The server class encapsulates the game's logic, ensuring a centralized and authoritative source for the game state.
    - Networking: Manages the communication between clients and ensures synchronization.
    - Game Logic: Responsible for enforcing game rules, determining outcomes, and updating clients.

GameClient.java (Client Class):

- Responsibilities:
    - Connects to the server to participate in the game.
    - Receives updates from the server regarding the game state.
    - Manages the local GUI, updating it based on server updates.
    - Sends user input (moves) to the server for processing.
        - Allows player functionality (like clicking a cell on GUI board to make a move)
- Why?
    - Encapsulation: Separates client-side operations from server-side operations, promoting modular design.
    - Networking: Handles communication with the server.
    - GUI Interaction: Manages the graphical representation and user interactions on the client side.

TicTacToeBoard.java:

- Responsibilities:
    - Represents the game board as a 3x3 grid.
    - Keeps track of the state of each cell (X, O, or empty).
    - Checks for win, loss, or draw conditions.
    - Records and retrieves moves made during the game.
- Why?
    - Encapsulation: Manages the game board's state and logic independently.
    - Game Logic: Enforces rules specific to the Tic Tac Toe game.

GameGUI.java:

- Responsibilities:
  - Implements the graphical user interface for the game.
  - Displays the Tic Tac Toe grid, game status, and player information.
  - Handles user interactions, such as clicks on cells and buttons.
- Why?
  - Encapsulation: Separates GUI concerns from game logic.
  - User Interaction: Manages the visual aspects and interactions, providing a responsive interface.
  - Modularity: Allows for changes in the GUI without affecting the underlying game logic.

LastMove.java:

- Responsibilities:
  - Stores information about the move, including the player and the chosen cell.
- Why?
  - Encapsulation: Bundles move-related information for easy transmission between server and clients.
  - Modularity: Provides a clean structure for representing player moves in the system.

HOW THE CLASSES ABOVE WILL WORK TOGETHER:

GameServer.java (Server Class):

- Has instances of TicTacToeBoard to manage the game state and enforce game logic.
- Manages instances of GameClient to communicate with connected clients.
- Receives moves from clients, updates the game state, and sends updates to clients.

GameClient.java (Client Class):

- Connects to an instance of GameServer.
- Manages the local GUI using an instance of GameGUI.
- Sends user input (player moves) to the server.
- Receives updates from the server and updates the local GUI accordingly.

TicTacToeBoard.java:

- Manages the state of the Tic Tac Toe board.
- Checks for win, loss, or draw conditions.
- Stores and retrieves moves made during the game.
- Is used by GameServer to enforce game logic.

GameGUI.java:

- Implements the graphical user interface.
- Handles user interactions and updates based on server updates.

- Is used by GameClient to manage the local GUI.

LastMove.java:

- Stores information about the move, including the player and the chosen cell.
- Is used for communication between GameServer and GameClient.

METHODS AND VARIABLES:

GameServer.java (Server Class):
Methods:

- startGame(): Initiates a new game, assigns symbols to players, and starts the game loop.
- processMove(move: Move): Receives and processes moves from clients, updating the game state.
- updateClients(): Sends updates to all connected clients about the current game state.
- checkWin(): Checks for win conditions on the game board.
- checkDraw(): Checks for a draw condition on the game board.

Variables:

- gameBoard: TicTacToeBoard: Instance to manage the game state and enforce game logic.
- players: List<Player>: List to keep track of connected players.
- (Other necessary variables for networking, synchronization, etc.)

GameClient.java (Client Class):
Methods:

- connectToServer(server: GameServer): Connects to the specified server.
- sendMove(move: Move): Sends user moves to the server for processing.
- updateGUI(): Updates the local graphical user interface based on server updates.

Variables:

- localServer: GameServer: Instance representing the connected server.
- clientID: int: Unique identifier for the client.
- gameGUI: GameGUI: Instance to manage the local graphical user interface.

TicTacToeBoard.java:
Methods:

- makeMove(player: Player, cell: Cell): boolean: Records a move on the game board.
- checkWin(): boolean: Checks for win conditions on the game board.
- checkDraw(): boolean: Checks for a draw condition on the game board.
- resetBoard(): Resets the game board for a new game.

Variables:

- boardState: array[3][3]: Represents the state of each cell on the game board.

Player.java:
Methods:

- (No specific methods mentioned, as player-related actions are likely managed by GameServer and GameClient.)

Variables:

- symbol: Symbol: Represents the assigned symbol for the player (X or O).

GameGUI.java:
Methods:

- updateBoard(): Updates the graphical representation of the game board.
- displayStatus(): Displays the current game status.
- displayInstructions(): Displays game instructions.
- promptForMove(): Prompts the user for a move.
- displayEndGame(): Displays the end-of-game information.

Variables:

- (Variables to manage GUI components, such as buttons, labels, etc.)

Move.java:
Variables:

- player: Player: Represents the player making the move.
- cell: Cell: Represents the cell on the game board where the move is made.